



Собственная система маршрутизации вызовов Ring-All («Smart Queue»)

Delivered by: ASRP · **Role:** ASRP owned the call-routing subsystem end to end (design, implementation, roadmap) · **Engagement type:** Independent contractor · **Domain:** Cloud telephony for freight logistics

Реализовано: ASRP · **Роль:** ASRP полностью владела подсистемой маршрутизации вызовов (проектирование, реализация, roadmap) · **Тип сотрудничества:** независимый подрядчик · **Домен:** облачная телефония для грузовой логистики

1. Краткое резюме

ASRP спроектировала и построила собственную систему распределения вызовов по принципу **ring-all** — внутреннее название «Smart Queue» — для облачной телефонной платформы, обслуживающей грузовую логистику. Когда звонок поступает на номер с включённой маршрутизацией, платформа **одновременно звонит всем залогиненным агентам организации. Первый агент, чья линия отвечает, забирает вызов**; все остальные линии сбрасываются, звонящий и агент-победитель соединяются, а если никто не отвечает — срабатывает настраиваемый резервный маршрут (fallback).

Отличительное инженерное свойство — **детерминированная гарантия «первый ответивший побеждает» при гонке**: два агента могут ответить с разницей в миллисекунды, и побеждает ровно один, без двойного ответа и без потери звонящего. Эта гарантия обеспечивается одним глаголом (verb) **Jambonz¹ dial** с массивом `target[]`, хуком подтверждения (confirm hook) на каждую цель и распределённой блокировкой **Redis** (Redlock), которая сериализует захват вызова. Весь путь закрыт двумя независимыми feature-флагами и опирается на собственную базу данных MySQL платформы — это намеренно **полностью собственный (in-house)** механизм, а не внешняя hunt-group PBX или сторонний ACD.

Smart Queue — это **пилот**, кандидат на замену действующей hunt-group внешней PBX платформы. Основная логика маршрутизации, захвата вызова, гейтинга, присутствия (presence) и fallback построены и покрыты автоматизированным набором тестов; главный фокус доработки перед переходом в продакшн — граничные случаи детекции ответа на исходящих carrier-маршрутах (см. §13–14). Примечательно, что нынешняя архитектура — уже **вторая**, которую ASRP построила для этой функции: изначальный прототип (POC) был влит в основную ветку, затем **полностью откачен**, а затем перестроен по другой архитектуре — история в §8.

2. Бизнес-контекст и проблема

Платформа маршрутизирует входящие звонки перевозчиков на агентов-людей, и у неё уже была собственная очередь — **первого поколения, по модели pull**. Звонок в очереди «парковался» и показывался всей организации в UI платформы; свободный агент **кликал, чтобы забрать** его, и платформа затем набирала этого агента и соединяла звонящего. Эта модель уже опиралась на блокировку Redis, так что если два агента кликали почти одновременно, побеждал ровно один. Модель работала для команды, следящей за экраном, но она принципиально **реактивна**: телефон сам по себе не звонит, поэтому звонок ждёт, пока человек его заметит и кликнет.

Бизнесу была нужна противоположная эргономика — **чтобы телефоны всех свободных агентов звонили одновременно** в момент поступления вызова, плюс живое присутствие (presence) и видимость для супервайзера — кто на звонке и как каждый звонок был маршрутизирован. Эта возможность была впервые реализована за счёт **внешней облачной PBX** hunt-group. Она давала поведение «звони всем и маршрутизируй», но выносила самые важные части — поведение очереди, присутствие агентов и то, кто «получил» звонок — **за пределы** платформы, где их было трудно наблюдать, расширять или сверять с собственными real-time UI платформы для агентов и супервайзеров. Присутствие во внешней PBX и присутствие в UI платформы стали двумя источниками истины, которые могли расходиться.

Задача состояла в том, чтобы вернуть это поведение ring-all **внутрь платформы** — сделать очередь платформно-нативной, которая:

- одновременно звонит всем доступным агентам (минимальное ожидание, без последовательного обзвона по списку),
- позволяет самому быстрому агенту побеждать **детерминированно**, без двойных ответов,
- управляется **собственной** базой данных и состоянием присутствия агентов платформы (единый источник истины),
- пушит статус в реальном времени в UI агента и супервайзера, и
- безопасно уходит в fallback, когда ни один агент не доступен.

3. Почему строили in-house (обоснование архитектуры)

Очевидной альтернативой было продолжать опираться на внешнюю PBX/ACD или взять готовый продукт контакт-центра. Рекомендация ASRP — строить очередь **внутри** платформы, и на это были конкретные причины, а не просто предпочтение:

- **Единый источник истины для присутствия.** Логин/логаут агента уже жили в платформе. Внешняя hunt-group ведёт *своё собственное* присутствие, из-за чего UI платформы и PBX расходятся. Собственная очередь читает то же состояние логина, что показывает UI.
- **Наблюдаемость и контроль.** Каждое решение по маршрутизации — кто был подходящим кандидатом, кому звонили, кто выиграл гонку, почему сработал fallback — это структурированное событие в собственных логах и базе данных платформы, а не непрозрачный результат от третьей стороны.
- **Семантика захвата вызова специфична.** «Позвонить всем, первый ответ побеждает, гарантированно ровно один победитель, остальных сбросить, fallback идемпотентно» — это точный контракт. Владение набором номера и блокировкой позволяет обеспечить и напрямую

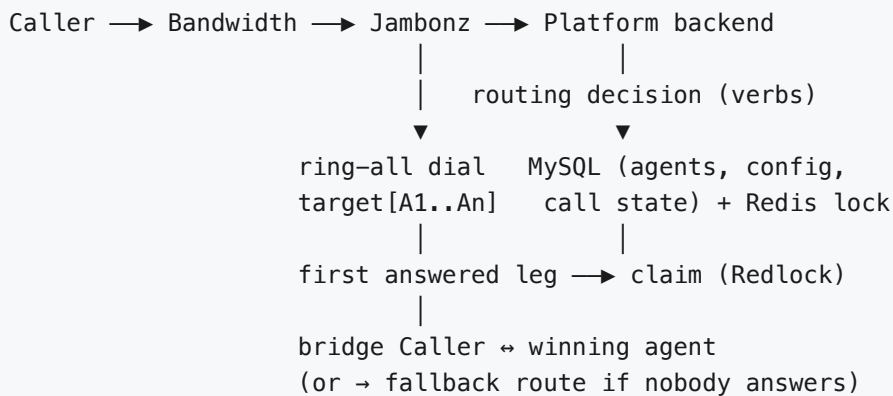
протестировать этот контракт; передача его PBX означает наследование того, что эта PBX делает по своему усмотрению.

- **Расширяемость.** Таймаут звонка, максимум агентов, включение по номеру, кастомные сообщения и будущая аналитика — всё это достижимо, потому что механизм является кодом, которым владеет команда.

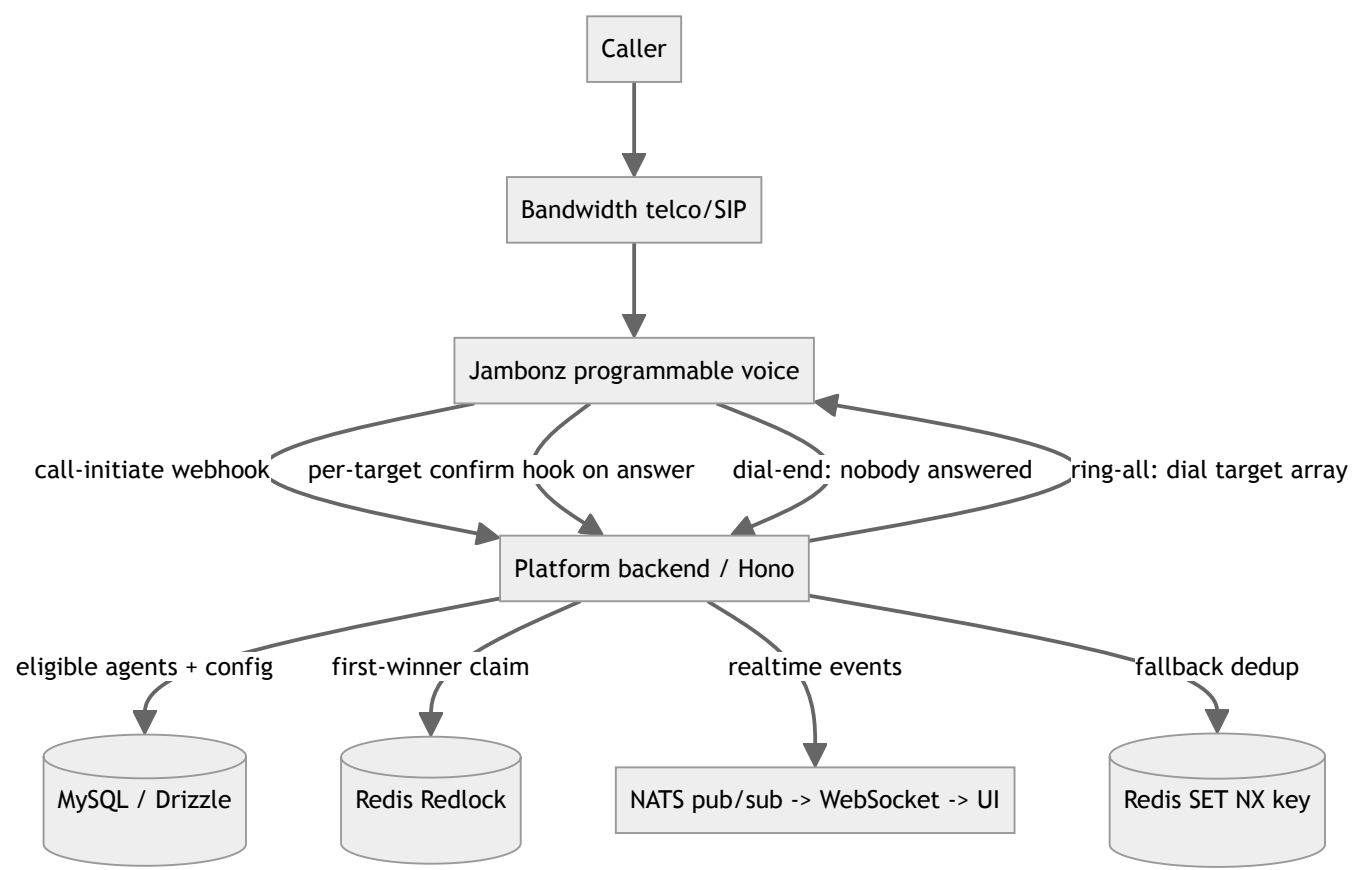
Компромисс — принятый осознанно — в том, что теперь платформа сама владеет сложными телефонными проблемами (гонки при одновременном ответе, супервизия ответа, идемпотентность fallback), которые PBX бы скрыла. §8 и §14 честно говорят о том, где это создаёт трудности.

4. Архитектура системы

Хребет вызова: Звонящий → Bandwidth (телефония/SIP) [^bandwidth-ru] → Jambonz (программируемая голосовая связь) → бэкенд платформы. Бэкенд принимает решение о маршрутизации для каждого звонка и возвращает verbs Jambonz; Jambonz выполняет ring-all dial и обращается обратно в бэкенд в каждой точке принятия решения (ответ, конец набора).



Роли компонентов



Компонент	Ответственность
Jambonz	Основной голосовой движок. Выполняет ring-all как единый verb dial с массивом целей target[] ; вызывает хук подтверждения для конкретной цели, когда линия отвечает; соединяет выигравшие линии; управляет fallback через action hook на конец набора.
Bandwidth	Телеком/SIP-оператор. Также поддерживаются вспомогательные verbs фоновой музыки (BXML).
Бэкенд платформы (TypeScript, веб-фреймворк Hono)	Решения по маршрутизации, право участия, захват вызова, fallback, feature-гейтинг, рассылка событий.
MySQL (Drizzle ORM)	Присутствие агентов, конфигурация маршрутизации по пользователю/номеру, состояние звонка в реальном времени.
Redis	Распределённая блокировка (Redlock) для гонки захвата; отдельный ключ для дедупликации fallback.
NATS + WebSockets	События очереди/захвата/статуса агента в реальном времени для UI агента и супервайзера.

5. Как работает маршрутизация

Ring-all — это **единый verb Jambonz dial с массивом target[]** и `confirmHook` на каждую цель. Подтверждение происходит **автоматически при ответе** через этот хук, без шага с нажатием клавиши для агента (почему — см. §8). Когда линия агента отвечает, бэкэнд получает **Redis Redlock**, привязанный к звонку, и внутри блокировки фиксирует первого ответившего как победителя и соединяет звонящего с ним; любой последующий ответ обнаруживает, что звонок уже захвачен, и его линия аккуратно завершается. Именно эта блокировка гарантирует ровно одного победителя, даже если два агента отвечают с разницей в миллисекунды. Если никто не отвечает в течение таймаута звонка, хук конца набора запускает **идемпотентный fallback-маршрут** (§11).

Полный пошаговый поток — фиксированный порядок маршрутизации, контрольные точки гейтинга, отбор подходящих кандидатов, `payload` хука на цель, точные ответы захвата/соединения/сброса линии и диаграмма последовательности вызова — поддерживается в инженерном справочнике ASRP. Более глубокие технические детали доступны по NDA.

6. Пример звонка простыми словами

Перевозчик набирает номер брокера с включённым Smart Queue. Два агента — назовём их **Прия** и **Сэм** — оба залогинены и подходят.

1. Звонок поступает. Бэкэнд проверяет два `feature-флага`, находит Прию и Сэма подходящими, помечает звонок как «в smart queue» и говорит Jambonz звонить **обоим** телефонам одновременно. Их UI загораются входящим звонком.
2. Оба телефона звонят. Прия тянется к своему телефону примерно на **полсекунды** раньше Сэма. Она отвечает.
3. Ответенная линия Прии вызывает её хук подтверждения. Бэкэнд берёт блокировку по этому звонку, видит, что он не захвачен, и записывает «Звонок захвачен Прией». Он возвращает пустой ответ, и Jambonz соединяет звонящего напрямую с Прией. Её UI переключается на «на звонке».
4. Чуть позже отвечает и Сэм. Срабатывает **его** хук подтверждения — но бэкэнд берёт блокировку, видит, что звонок уже захвачен Прией, и возвращает команду сброса. Линия Сэма аккуратно обрывается; его UI очищается. Он никогда не слышит тишину и не говорит поверх Прии.
5. Звонящий и Прия теперь разговаривают. Со стороны звонящего это выглядело как один звонок и снятие трубки — они никогда не узнали, что звонило два телефона.

Если бы ни Прия, ни Сэм не ответили в течение таймаута звонка, набор завершился бы без соединения, `fallback-путь` сработал бы ровно один раз (защищённый от дублирующихся `webhook'ов`), и звонящий был бы направлен на настроенное резервное направление — либо услышал бы вежливое сообщение «не удаётся соединить», если оно не настроено.

7. Что построила ASRP

- **Механизм ring-all dial** как `verbs` программируемой голосовой связи Jambonz: единый `verb dial`, нацеленный на массив адресатов-агентов, каждый со своим хуком подтверждения.

- **Захват вызова «первый ответивший»**, защищённый Redis Redlock, так что ровно один агент побеждает, а остальные линии сбрасываются.
- **Двухуровневый feature-гейт** (флаг организации + флаг по номеру), проверяемый согласованно при входе звонка, при захвате и на пути fallback.
- **Присутствие агентов**: логин/логаут на основе базы данных с переключателем самообслуживания для агентов и представлением управления для супервайзеров, плюс архив истории логинов.
- **События в реальном времени**, отправляемые в UI через NATS pub/sub и WebSockets (очередь инициирована, звонок захвачен, статус агента).
- **Безопасный fallback**: идемпотентный путь, который направляет звонящего на настроенное резервное направление или проигрывает вежливое сообщение, если оно не настроено.
- **Процедуры tRPC** и переключатели UI для логина агента и управления агентами супервайзером.

8. Инженерная история: очередь, которую выпустили, убили и перестроили

Самая поучительная часть этого проекта в том, что архитектура из §4–5 — это **не** первая архитектура, которая была построена и влита в основную ветку.

Проблема. Спроектировать очередь, которая звонит агентам и позволяет одному из них принять звонок детерминированно.

Первая попытка (РОС, влита). Изначальный прототип решал это через **конференц-мост плюс явное «Нажмите 1, чтобы принять»**: вызываемые агенты присоединялись к конференции, и агент нажимал клавишу, чтобы подтвердить, что он берёт звонок. Он поставлялся со своими собственными маршрутами подтверждения и исходящими маршрутами, отдельной таблицей событий, миграцией базы данных и большим (~2400 строк) проектным документом. Он был завершён и **влит в основную ветку**.

Разворот. Весь этот merge был затем **полностью откачен** — сервис, оба маршрута подтверждения, исходящий маршрут, миграция, таблица событий и проектный документ были удалены одним ревертом, и кодовая база вернулась к состоянию без какого-либо Smart Queue. Причина была решением по срокам поставки, а не техническим тупиком: та же потребность в ring-all-и-маршрутизации уже **удовлетворялась на пути внешней PBX**, построенном параллельно (§2), поэтому вместо продолжения доработки совершенно нового собственного голосового механизма команда выпустила решение на PBX и **отложила** собственную очередь. Её отложили, а не забросили.

Перестройка (текущая архитектура). Собственный подход был позже **возрождён** и перестроен с нуля по намеренно другой архитектуре: без конференц-моста и **без шага с клавиатурой**. Вместо этого — единый dial с массивом target[], хук подтверждения на каждую цель и захват, сериализованный через Redlock (§5). Подтверждение стало **автоматическим при ответе** — самый быстрый ответ забирает блокировку — потому что принуждение каждого агента «нажать 1» добавляло трение на быстром пути, который очередь и призвана оптимизировать. Модель данных сохранили намеренно скромной: перестройка легла поверх существующей таблицы конфигурации

pull-очереди первого поколения и её захвата через блокировку Redis (§2), добавив **ноль новых колонок** в эту таблицу и введя только таблицы логина агента и истории логинов, которые действительно были нужны.

Честный поворот. Убрать «Нажмите 1» — правильное решение для UX агента, но именно это и делает супервизию ответа сложной: без нажатия клавиши человеком система выводит «человек ответил» из сигнала ответа оператора связи — а некоторые маршруты сигнализируют «отвечено» до того, как человек реально взял трубку. Это главный пункт доработки перед продакшном (§14). Команда знает, что отброшенная конструкция обошла бы эту проблему стороной, и всё равно выбрала лучший UX, что точно формулирует оставшуюся работу, а не делает вид, что её нет.

9. Технологический стек

Слой	Технология
Программируемая голосовая связь / медиа	Jambonz (ring-all dial с target[] , хук подтверждения на цель, action hook на конец набора)
Телефония / SIP	Bandwidth (также используется для вспомогательного BXML hold-audio)
Бэкенд	TypeScript на веб-фреймворке Hono , работающий на рантайме Bun
Хранение данных	MySQL через Drizzle ORM
Распределённая координация	Redis — Redlock для гонки захвата; ключ SET NX для дедупликации fallback
Реальное время	NATS pub/sub → сервер WebSocket → UI агента/ супервайзера
API-поверхность для UI	Процедуры tRPC (статус, переключатель логина, управление агентами супервайзером)

О базе данных: MySQL, доступ через **Drizzle ORM** на TypeScript из общего пакета схемы. Платформа использует **стратегию с двумя видами подключения**, выбираемую по окружению — **serverless-драйвер PlanetScale** к MySQL в продакшне и драйвер **mysql2** к локальной MySQL (MySQL 9) для разработки — оба используют одну и ту же схему Drizzle, при этом чтения можно направлять на реплику.

10. Модель данных

Функция опирается на реляционную схему MySQL платформы (Drizzle ORM). Её отличительная черта — сдержанность: перестройка **переиспользовала таблицу конфигурации маршрутизации по агенту из pull-очереди первого поколения без добавления каких-либо колонок** и ввела только те таблицы, которые действительно были нужны — **состояние логина агента** (кто сейчас залогинен и, следовательно, подходит для вызова) и **архив истории логинов** (живая строка переносится сюда при логaute). Флаг включения по номеру живёт в конфигурации телефонного номера, а центральная таблица состояния звонка платформы несёт поля маршрутизации, которые заполняет эта функция

(статус, захвативший пользователь, id соединённой линии + время ответа, флаг возможности захвата, отметка времени постановки в очередь). Конфигурация по организации и по номеру хранится как **JSON, а не колонки**.

Полная таблица-за-таблицей схема — ключи, индексы и форма JSON-конфигурации — поддерживается в инженерном справочнике ASRP. Более глубокие технические детали доступны по NDA.

11. Надёжность и эксплуатация

- **Двухуровневый feature-гейт, применяемый повсеместно.** И флаг организации, и флаг по номеру должны быть истинными, и гейт перепроверяется в трёх независимых точках — при входе звонка, при подтверждении захвата и при fallback — так что изменение конфигурации в середине звонка не может «застрять» звонок на несогласованном пути. Отказы логируются с чёткой причиной.
- **Детерминированный захват при гонке.** Redis Redlock, привязанный к звонку, сериализует конкурирующие ответы; проверка «уже захвачено» внутри блокировки делает второго победителя невозможным. Проигравшие линии аккуратно сбрасываются.
- **Идемпотентный fallback.** Путь fallback защищён ключом Redis SET NX (TTL 2 часа), так что дублирующиеся или повторные webhook'и конца набора маршрутизируют звонящего максимум один раз. Если резервное направление не настроено, звонящий слышит вежливое сообщение вместо тишины, а отсутствующий маршрут логируется.
- **Изящное поведение при отсутствии агентов.** Звонок может быть помечен как доступный для захвата, даже если ни один агент не залогинен, что позволяет ручной захват из UI вместо жёсткого отказа.
- **Структурированное логирование жизненного цикла.** Каждый этап генерирует структурированное событие (инициация набора, набор инициирован, подтверждение принято/отклонено, захвачено, маршрутизация fallback/пропущена, не удалось соединить, очистка при обрыве звонящим) для трассируемости.

12. Параметры конфигурации

Поведение маршрутизации управляется явными, **настраиваемыми** параметрами, а не жёстко заданными константами — количество агентов, которым звонят на звонок, таймаут звонка, срок аренды блокировки захвата и окно дедупликации fallback — всё это настраивается по организации или по номеру, с разумными значениями по умолчанию. Это конфигурационные значения по умолчанию, проверенные по исходникам поставки, а **не** измеренные показатели производительности — реальных метрик на живом трафике пока не собрано (см. §13). Точные значения по умолчанию зафиксированы в инженерном справочнике ASRP (доступен по NDA).

13. Результаты и итоги

Путь ring-all, захват первым ответом, feature-гейт, присутствие агентов (логин/логаут/история), события реального времени и fallback — всё реализовано и покрыто автоматизированным набором тестов. Набор охватывает значимые сценарии: нет залогиненных агентов → удержание для ручного

захвата; один агент → захват через хук подтверждения; два агента в гонке → побеждает ровно один; сбой набора → fallback; звонящий обрывает звонок во время дозвона → чистая очистка без захвата и без fallback.

Smart Queue позиционируется как **пилот и кандидат на замену действующей hunt-group внешней PBX**. На момент проекта она **не** была измерена на продакшн-объёме звонков, поэтому эта страница не делает заявлений о времени до ответа, доле сброшенных звонков или загрузке агентов по сравнению со старой очередью — этих цифр пока не существует, и лучше сказать об этом прямо, чем их выдумывать.

14. Зрелость и roadmap

Честное разделение того, что уже выпущено, и того, что только спроектировано:

LIVE / построено и протестировано (за feature-гейтом): ring-all dial, детерминированный захват первым ответом под Redlock, двухуровневый feature-гейт, присутствие агентов на основе базы данных (логин/логаут/история), события очереди/захвата/статуса в реальном времени, идемпотентный fallback и путь ручного захвата, когда ни один агент не залогинен. Всё покрыто автоматизированным набором тестов.

Главный фокус доработки перед переходом в продакшн: граничные случаи детекции ответа на исходящих carrier/PBX-маршрутах. Поскольку подтверждение автоматическое при ответе (без клавиатуры — см. §8), переход в состояние «соединено» должен соответствовать *реальному* ответу человека на каждом типе маршрута; сделать это надёжным на всех carrier- и PBX-маршрутах — главный пункт, отделяющий пилот от продакшна.

Спроектировано / roadmap (следующие шаги): - **UI конфигурации** — включение, назначение групп, таймаут звонка, максимум агентов и fallback сегодня управляются через базу данных/JSON; UI самообслуживания для администраторов — естественный следующий шаг. - **Устойчивая история событий очереди** — выделенное хранилище событий (попытавшиеся цели, результаты звонка/неответа/занято, гонки захвата, метрики доли ответов) для аналитики и дашбордов наблюдаемости за пределами сегодняшнего структурированного логирования. - **Паритет по супервайзерам / устройствам** — селектор устройства, статус регистрации, последнее выбранное устройство и присутствие «на звонке» по устройству, чтобы достичь полного паритета с действующей внешней очередью. - **Доработка прогрессии состояний** — более детальная прогрессия жизненного цикла, отображаемая в UI (входящий → ожидание → соединено → завершено) для всех вариантов обрыва звонка и fallback.

15. Что реализовала ASRP

ASRP владела **всей подсистемой Smart Queue** поверх существующей модели звонков платформы — verbs ring-all, запросы на право участия, сервис захвата, feature-гейт, схему и CRUD логина агента, события реального времени, процедуры tRPC и переключатели UI.

- Спроектировала и реализовала собственную **маршрутизацию ring-all** (единый Jambonz dial с массивом `target[]` + хук подтверждения на цель).
- Построила **захват «первый ответивший»** с Redis Redlock и **идемпотентный fallback** (Redis `SET NX`).

- Разработала **двухуровневый feature-гейт** (флаги организации + по номеру), применяемый при входе, захвате и fallback.
- Спроектировала **модель данных присутствия агента** (состояние логина + архив истории) и её CRUD, а также запрос на право участия, соединяющий группы, конфигурацию маршрутизации по агенту и состояние логина.
- Реализовала **события WebSocket/NATS в реальном времени**, а также **процедуры tRPC и переключатели UI** для самостоятельного логина агента и управления супервайзером.
- Владела полным жизненным циклом **РОС → откат → перестройка**, включая осознанный редизайн от прототипа с конференц-мостом/подтверждением клавиатурой к модели автоматического захвата на блокировке (§8).
- Написала **автоматизированный набор тестов**, покрывающий основные сценарии маршрутизации.

16. FAQ (для разговоров с клиентами)

В: Это внешняя PBX или сторонний ACD? Нет — это полностью собственная очередь, построенная на verbs Jambonz плюс собственной базе данных платформы и Redis. Она была построена специально для того, чтобы перенести логику очереди, присутствие агентов и поведение захвата внутрь платформы, а не делегировать их внешней hunt-group PBX.

В: Агент нажимает клавишу, чтобы принять звонок? Нет. Все залогиненные агенты звонят одновременно, а подтверждение происходит автоматически при ответе через хук подтверждения на цель. Побеждает самый быстрый ответ. Более ранний прототип использовал «Нажмите 1, чтобы принять»; он был откачен, и механизм был перестроен без шага с клавиатурой (§8).

В: Что не даёт двум агентам получить один и тот же звонок? Распределённая блокировка Redis (Redlock), привязанная к звонку. Первая ответившая линия захватывает его внутри блокировки; любая более поздняя линия обнаруживает, что звонок уже захвачен, и сбрасывается. Ровно один победитель, детерминированно.

В: Что происходит, если ни один агент не отвечает? Идемпотентный путь fallback направляет звонящего на настроенное резервное направление; если оно не настроено, звонящий слышит вежливое сообщение «не удаётся соединить». Дублирующие или повторные webhook'и не могут маршрутизировать дважды.

В: Как включить или выключить это? Два независимых флага — один на уровне организации и один по номеру телефона. Оба должны быть включены, и гейт перепроверяется при входе звонка, при захвате и при fallback.

В: Какую базу данных вы использовали? MySQL через Drizzle ORM на TypeScript. Продакшн использует serverless-драйвер PlanetScale; локальная разработка использует драйвер `mysql2` к MySQL. Redis обеспечивает блокировку захвата и дедупликацию fallback.

В: Это в продакшне? Это пилот — кандидат на замену действующей hunt-group внешней PBX — с доработкой детекции ответа как главным пунктом перед переходом в продакшн. Реальных метрик трафика пока не собрано (§13).

В: Как это тестируется? Автоматизированный набор покрывает ключевые сценарии: нет залогиненных агентов (удержание для ручного захвата), один агент (захват через хук подтверждения), два агента в гонке (побеждает ровно один), сбой набора (fallback) и обрыв звонящим во время дозвона (чистая очистка).

Анонимизированное описание проекта, подготовленное ASRP для портфолио. Имя клиента, проприетарные пути в исходном коде, внутренние идентификаторы и содержание нерешённых дефектов/реестра рисков намеренно опущены в тексте. Более глубокие технические детали доступны по NDA.

1. **Jambonz** — платформа программируемой голосовой связи / CPaaS с открытым исходным кодом, которая связывает телефонию (SIP/PSTN операторов) с приложениями, выступая в роли медиа-шлюза между телефонной сетью и бэкендом. <https://jambonz.org> ↩